

머신러닝 5 교시

— 분류의 평가 지표 —

(Confusion Matrix · Precision · Recall · F1 · ROC-AUC)

학습 목표

- 정확도(Accuracy)만으로 분류 모델을 평가할 때 생기는 한계를 설명할 수 있다.
- 혼동 행렬(Confusion Matrix)을 이해하고 TP·FP·FN·TN 을 구분할 수 있다.
- 정밀도(Precision), 재현율(Recall), F1 점수의 의미와 사용 상황을 구분할 수 있다.
- 결정 임계값(threshold)을 조정해 ROC 곡선과 AUC 를 이해한다.
- 사이킷런으로 위 모든 지표를 직접 계산하고 시각화할 수 있다.

1. 왜 정확도(Accuracy)만으로는 부족한가

지금까지 우리는 모델의 성능을 단순히 정확도(accuracy)로만 평가했다. 정확도는 "전체 데이터 중 맞힌 비율"이라는 가장 직관적인 지표이지만, 실제 현장에서는 정확도만 보는 것이 매우 위험할 수 있다. 그 이유를 먼저 살펴보자.

1.1 정확도의 함정 — 클래스 불균형 사례

어떤 회사의 신용카드 거래 데이터를 떠올려보자. 100,000 건의 거래 중 진짜 부정거래(사기)는 100 건이다. 즉 정상인 99,900 건, 사기가 100 건으로 99.9% : 0.1%의 극단적인 불균형이다. 이때 "무조건 정상이라고 답하는" 가짜 모델을 만든다면, 그 모델의 정확도는 다음과 같이 계산된다.

```
정확도 = 맞힌 개수 / 전체 개수
        = 99,900 / 100,000
        = 0.999 → 99.9%
```

아무 일도 하지 않는 모델이 99.9% 정확도를 자랑하는 셈이다. 그러나 이 모델은 사기를 단 한 건도 잡지 못하므로 실무적으로는 완전히 무용하다. 이 사례가 보여주는 것은, "잘 맞히는가?"는 말의 의미를 좀 더 생각해 볼 필요가 있다는 것이다.

1.2 "좋은 모델"의 정의

어떤 분류 문제든 잘 맞히는 것에도 두 가지가 있고, 틀리게 맞히는 것도 두가지가 있다. (홍길동)

- 실제로는 음성인데 양성이라고 잘못 예측 — 거짓 양성(False Positive, FP)
- 실제로는 양성인데 음성이라고 놓침 — 거짓 음성(False Negative, FN)

이 두 오류의 "비용"은 도메인마다 다르다. 어느 쪽이 더 치명적인지를 먼저 정해야 평가 지표를 고를 수 있다. 아래 그림은 의학적 검사에서 정상군과 환자군의 점수 분포가 겹치는 영역에서, 어디에 임계값(검은 수직선)을 두느냐에 따라 거짓 양성·거짓 음성의 비율이 어떻게 달라지는지 보여준다.

도메인별로 어느 오류가 더 치명적인지 정리하면 다음과 같다.

분류 오류의 비용 비교

— 의료 진단 (암 검진)		
	— FN(실제 암을 놓침)	← 매우 치명적
	— FP(정상을 암으로 의심)	← 추가 검사로 회복 가능
— 스팸 필터		
	— FN(스팸을 못 거름)	← 사용자 짜증
	— FP(정상 메일을 스팸 처리)	← 중요 메일 손실, 더 치명
— 신용카드 부정거래 탐지		
	— FN(사기를 놓침)	← 직접 손실
	— FP(정상 거래를 사기 의심)	← 고객 불편

따라서 "좋은 분류 모델"이라는 말은 도메인마다 다르게 정의되며, 우리는 그 정의에 맞는 지표를 골라야 한다. 이를 위해 먼저 모든 지표의 출발점인 혼동 행렬(Confusion Matrix)을 만들게 된다.

2. 혼동 행렬(Confusion Matrix)

혼동 행렬은 분류 모델의 예측 결과를 "실제 정답"과 "모델 예측"의 조합으로 정리한 결과표다.

이진 분류의 경우 결과는 다음 네 가지 중 하나가 된다.

2.1 TP / FP / FN / TN — 그림으로 이해하기

첫번째 어려움: TP, FP는 "맞혔는가", 뒷 글자 Poitive/Negative은 "무엇이라고 예측했는가"를 뜻한다. 즉 FP는 "틀렸는데 양성이라 예측", FN은 "틀렸는데 음성이라 예측"이다.

우리말로 해석하기에는 뒤에 부분을 먼저 말해서 "양성이라 예측했는데 틀렸다(FP)" "음성이라 예측했는데 맞았다(TN)"로 하는 것이다.

TP : True Positive - “

FP : False Positive - “

TN : True Negative - “

FN : False Negative - “

아래 그림은 혼동 행렬의 네 칸이 어떻게 배치되는지를 시각적으로 보여준다. 행은 "실제 클래스(actual)", 열은 "예측 클래스(predicted)"이며, 각 칸의 명칭과 의미가 함께 표시되어 있다. "양성(positive)"은 우리가 관심 있는 클래스(예: 암, 사기, 스팸)를 뜻한다.

		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

2.2 사이킷런의 혼동 행렬 배치

사이킷런 `confusion_matrix(y_true, y_pred)` 함수는 다음과 같이 2×2 형태의 numpy 배열로 결과를 반환한다.

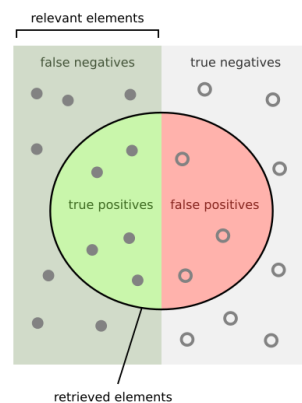
```

          예측 음성   예측 양성
실제 음성 [  TN      ,   FP  ]
실제 양성 [  FN      ,   TP  ]
  
```

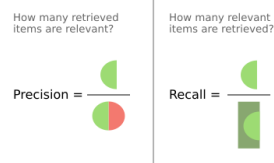
행이 실제 클래스, 열이 예측 클래스라는 점에 주의한다. 다른 라이브러리는 행/열이 반대인 경우도 있어 헷갈리기 쉽다. 위 그림 [그림 2]에서의 시각적 배치와 사이킷런의 인덱스 순서를 함께 익혀두면 좋다.

3. 핵심 지표 — 정밀도, 재현율, F1

혼동 행렬의 네 칸 TP·FP·FN·TN 으로부터 가장 자주 쓰이는 세 가지 지표가 만들어진다. 셋 모두 "양성(positive)"을 어떻게 다루는지에 초점을 둔 지표라는 공통점이 있다. 아래 다이어그램은 정밀도와 재현율이 무엇을 "분모"로 두고 무엇을 "분자"로 두는지를 한 장의 그림으로 보여준다.



[그림 3] 정밀도(Precision)와 재현율(Recall)의 시각적 정의. 위쪽 원은 "실제 양성(actual positive)", 아래쪽 사각형은 "모델이 양성이라 예측한 것(selected)"을 뜻한다. 둘의 교집합이 TP, 비율의 분모를 무엇으로 두느냐에 따라 정밀도와 재현율이 갈린다.



3.1 정밀도(Precision) – "양성이라 말했을 때 진짜 양성이었나"

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

정밀도는 모델이 "양성"이라고 분류한 것들 중 실제로 양성이었던 비율이다. 양성으로 외친 결과의 "신뢰도"라고 볼 수 있다. 잘못된 양성 판정의 비용이 클 때(예: 정상 메일을 스팸으로 처리, 정상 거래를 사기로 차단) 정밀도를 우선시한다. 그림 3 에서 아래쪽 사각형 안의 "진짜 양성(녹색)"의 비율을 떠올리면 된다.

3.2 재현율(Recall, Sensitivity) – "진짜 양성을 얼마나 놓치지 않았나"

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

재현율은 실제 양성 중 모델이 양성으로 잡아낸 비율이다. "얼마나 빠뜨리지 않고 잡았는가"를 본다. 양성을 놓치는 비용이 클 때(예: 암 검진에서 환자를 놓침, 사기를 놓침) 재현율을 우선시한다. 그림 3 에서 위쪽 원 안의 "발견된(녹색)" 비율을 떠올리면 된다.

3.3 정밀도와 재현율은 보통 트레이드오프 관계

두 지표는 서로 끌어당기는 관계다. 모델이 양성을 더 "적극적으로" 외칠수록 재현율은 올라가지만 거짓 양성도 함께 늘어 정밀도는 떨어진다. 반대로 "보수적"으로 양성을 외치면 정밀도는 올라가지만 놓치는 양성이 많아 재현율이 떨어진다.

이 트레이드오프는 다시 [그림 1] 의 분포 그림으로 돌아가 생각하면 명확하다. 임계값(검은 세로선)을 왼쪽으로 옮기면 양성 판정 영역이 넓어지므로 "환자" 분포에서 잡아내는 비율(재현율)은 증가하지만, "정상" 분포에서 빨려 들어오는 부분(거짓 양성)도 같이 늘어 정밀도는 줄어든다. 다시 생각해 보면, "암이다"고 해서 예측이 틀리는 것과 "암이 아니다"고 해서 예측이 틀리는 것이 동시에 빈도가 올라가기는 어렵다는 뜻이다.

임계값 조정에 따른 변화

└─ 임계값 ↓ (양성을 더 많이 외침)
| └─ 재현율 ↑
| └─ 정밀도 ↓
└─ 임계값 ↑ (양성을 보수적으로 외침)
| └─ 재현율 ↓
| └─ 정밀도 ↑

3.4 F1 점수 – 두 지표의 조화평균

$$\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

정밀도와 재현율 모두 어느 정도 균형 있게 좋길 원할 때 사용한다. 산술평균이 아닌 조화평균을 쓰는 이유는, 둘 중 하나가 0 에 가까워지면 F1 도 함께 0 에 가까워져 "한쪽만 잘하는 모델"을 걸러내기 위함이다.

3.5 어떤 지표를 봐야 할까

상황별 추천 지표

- └ 클래스 균형이 잘 맞고 둘 다 비슷히 중요 → Accuracy 또는 F1
- └ FP의 비용이 큼 (정상→양성 오인이 치명) → Precision
- └ FN의 비용이 큼 (양성을 놓치면 치명) → Recall
- └ 두 지표 균형 + 클래스 불균형 → F1, ROC-AUC, PR-AUC

3.5 도메인별로 다르다는 확인

의료분야에서는 FP가 높아도 좋다. FN이 높은 것이 가장 피해야 하는 것이다. 즉,

Precision(정밀도)이 낮아도 좋고 반대로 Recall(재현율)은 높아도 좋다.

재판에서는 FP가 높으면 안된다. 즉, FN은 높아도 감수할 수 있겠다. 따라서 Precision은 높아야 하고, Recall 값은 낮아도 감수할 것이다.

이와 같이 평가 기준은 도메인별로 다른 기준을 보아야 한다.

4. 다중 분류에서의 평가 지표

정밀도·재현율·F1은 본래 이진 분류용으로 정의되지만, 다중 분류에서도 쓸 수 있다. 다만 "각 클래스별로 따로 계산한 뒤 어떻게 합칠 것인가"에 대한 선택이 필요하다. 사이킷런의 average 파라미터에서 이 선택을 한다.

4.1 평균 방식 세 가지

다중 분류의 평균 방식

- └ macro : 각 클래스별 지표를 단순 평균 (클래스 수로 나눔)
 - └ 모든 클래스를 동등하게 본다. 소수 클래스도 한 표.
- └ micro : 전체 TP·FP·FN을 모은 뒤 한꺼번에 계산
 - └ 결과적으로 다중분류에서는 정확도와 같다.
- └ weighted : 클래스 빈도(샘플 수)로 가중 평균
 - └ 다수 클래스의 성능에 결과가 쏠려간다.

4.2 어떤 평균을 쓸까

- 클래스 불균형이 있고 "소수 클래스도 잘 맞는지"를 보고 싶다면 → macro
- 불균형 상황에서 "전체 데이터 기준 성능"을 보고 싶다면 → weighted
- 정확도와 같은 의미로 한눈에 보고 싶다면 → micro

위의 [9 단계] 실습에서 iris(붓꽃) 다중 분류에 세 가지 평균을 모두 적용해 비교한다. 균형 데이터인 Iris는 셋의 차이가 거의 없지만, 불균형 데이터에서는 차이가 매우 커진다.

5. 결정 임계값(Threshold)과 ROC, AUC

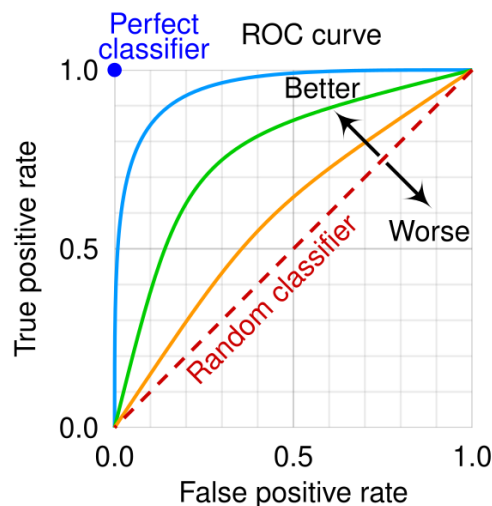
대부분의 분류 모델은 실제로는 "확률"을 출력하고, 0.5 를 기준(threshold)으로 양성/음성을 가른다. 이 0.5 라는 기준값은 절대적인 것이 아니며, 도메인 요구에 따라 자유롭게 바꿀 수 있다. 임계값을 바꾸면 정밀도와 재현율이 어떻게 변하는지 한눈에 보는 도구가 ROC 곡선이다.

5.1 ROC 곡선이란

ROC(Receiver Operating Characteristic) 곡선은 임계값을 0 에서 1 까지 변화시키면서 두 값을 그린 곡선이다.

- X 축: 거짓 양성 비율(False Positive Rate, FPR) = $FP / (FP + TN)$
- Y 축: 진짜 양성 비율(True Positive Rate, TPR) = Recall = $TP / (TP + FN)$

이상적인 모델은 (0, 1) — 즉 거짓 양성은 0 이면서 진짜 양성은 모두 잡는 — 모서리에 곡선이 가까이 붙는다. 무작위로 찍는 모델은 대각선($y = x$) 근처를 그린다. 아래 그림은 좋은 모델일수록 곡선이 좌상단에 "휘어 올라가는" 모습을 잘 보여준다.



[그림 4] ROC 곡선 예시 — 좌상단 모서리에 가까울수록 좋은 모델. 대각선은 무작위 추정 수준의 성능

5.2 AUC — 곡선 아래 면적

AUC(Area Under the Curve)는 ROC 곡선 아래 면적이다. 이상 모델은 1.0, 무작위 모델은 0.5 에 가깝다. 그림 4 에서 위쪽 곡선의 아래 면적이 크고, 대각선의 아래 면적은 정확히 0.5 이다.

AUC 값 해석 가이드

0.90 ~ 1.00	매우 우수
0.80 ~ 0.90	우수
0.70 ~ 0.80	충만함
0.60 ~ 0.70	약함
0.50	근방 무작위 수준 (의미 없음)

5.3 ROC vs PR(Precision-Recall) 곡선

클래스가 매우 불균형한 경우에는 ROC 곡선이 실제보다 모델을 좋아 보이게 만들 수 있다. 진짜 양성(positive)이 워낙 적어서 FPR 이 거의 변하지 않기 때문이다. 이런 상황에서는 정밀도-재현율 곡선(PR Curve)과 그 아래 면적(PR-AUC, Average Precision)을 함께 보는 것이 좋다.

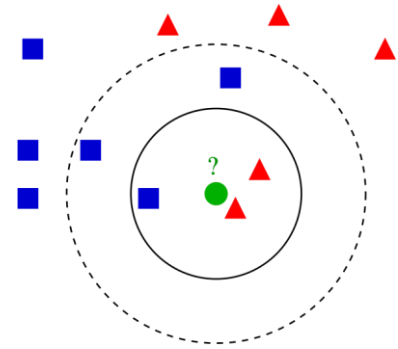
곡선 선택 가이드

└─ 클래스 균형이 어느 정도 맞음 → ROC-AUC
└─ 양성 비율이 매우 작은 불균형 데이터 → PR-AUC 함께 보기

6. 사이킷런(scikit-learn) 코드 실습

이번 실습은 위스콘신 유방암(breast cancer) 데이터셋을 이용한다. 569 개 샘플, 30 개 특성으로 구성된 이진 분류 데이터로, 클래스가 양성(benign, 357 개)과 악성(malignant, 212 개)으로 나뉜다. 의료 도메인이라 "양성을 놓치지 않는 것"이 특히 중요한 데이터다. 모델은 1 교시에서 다룬 KNN(K-Nearest Neighbors)을 그대로 사용한다.

[그림 5] KNN의 동작 — 새 점(별 모양)에 가장 가까운 K 개의 이웃을 보고 다수결로 클래스를 결정한다. K 가 작으면 노이즈에 민감하고, K 가 크면 결정 경계가 부드러워진다.



실습 환경 준비

```
# 필요한 라이브러리 설치 (터미널에서)
pip install scikit-learn pandas matplotlib
```

[1 단계] 데이터 불러오기

```
from sklearn.datasets import load_breast_cancer
import pandas as pd

data = load_breast_cancer()
X, y = data.data, data.target

print("특성 개수 :", X.shape[1])
print("전체 샘플 :", X.shape[0])
print("클래스 이름:", data.target_names) # ['malignant' 'benign']
print("클래스 분포:", pd.Series(y).value_counts().to_dict())
```

주의할 점: 사이킷런 breast_cancer 데이터에서는 0 이 악성(malignant), 1 이 양성(benign)이다. 우리가 "양성(positive)"이라고 부르는 머신러닝 용어와 의학의 "양성(benign)"이 헷갈릴 수 있다. 이 실습에서는 "악성(malignant) = 1" 로 재라벨링하여 "악성을 양성 클래스(positive class)"로 다룬다. 그래야 "악성을 놓치지 않는다"가 재현율의 의미와 일치한다.

```
# 악성을 1, 양성을 0 으로 재라벨링
y = (y == 0).astype(int) # 0(malignant) → 1, 1(benign) → 0
print("양성(=악성, 1) 비율:", y.mean())
```

💡 [코멘트] 보통 머신러닝에서는 우리가 찾아내고자 하는 치명적이거나 중요한 정답(암, 스팸, 불량품)을 1(Positive)로 둔다. 이 데이터셋은 악성이 0으로 되어 있어서, 직관적인 계산을 위해 악성을 1로 바꿔주는 센스가 필요하다.

[2 단계] 훈련 / 테스트 분리 + KNN 모델 학습

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# KNN은 거리 기반이므로 스케일링이 거의 필수
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_s, y_train)

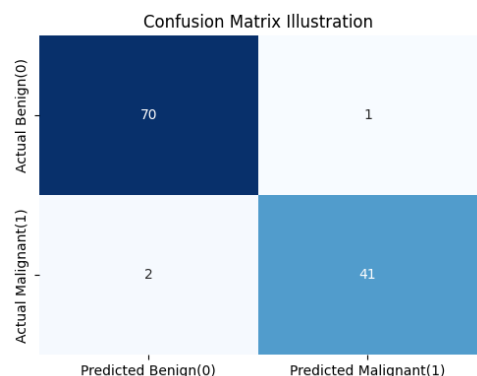
y_pred = knn.predict(X_test_s)
```

[3 단계] 혼동 행렬 구하기

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)
print("혼동 행렬:")
print(cm)

# 사이킷런 배열 풀어 읽기
tn, fp, fn, tp = cm.ravel()
print(f"TN={tn}, FP={fp}, FN={fn}, TP={tp}")
```



출력 예 (random_state=42 기준): $TN \approx 70$, $FP \approx 1$, $FN \approx 2$, $TP \approx 41$. 즉 "실제 악성을 음성으로 놓친(FN) 건수"를 직접 확인할 수 있다. 위 [그림 2]의 4 칸과 비교하며 어떤 칸의 숫자인지 짚어보자.

[4 단계] 정확도 · 정밀도 · 재현율 · F1 직접 계산

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score
)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

```
print(f"정확도   Accuracy : {acc :.4f}")
print(f"정밀도   Precision: {prec:.4f}")
print(f"재현율   Recall   : {rec :.4f}")
print(f"F1 점수   F1       : {f1  :.4f}")
```

[5 단계] classification_report – 한 번에 보기

```
from sklearn.metrics import classification_report

print(classification_report(
    y_test, y_pred,
    target_names=["benign(0)", "malignant(1)"]
))
```

출력은 클래스별 precision/recall/f1 과 함께 macro/weighted avg 가 함께 나와, 다중 분류로 그대로 확장 가능한 형식이다.

💡 [코멘트] classification_report 결과가 나오면 가장 먼저 malignant(1) 행의 recall(재현율)을 확인한다. 이 값이 우리가 암 환자를 얼마나 잘 찾아냈는지 알려주는 가장 핵심 지표이기 때문이다.

[6 단계] 임계값 조정 – "악성을 더 적극적으로 잡자"

의료 도메인에서는 악성을 놓치는 비용이 크므로, 모델이 "악성일 확률 ≥ 0.3 "이면 악성으로 판정하도록 임계값을 낮춘다. predict() 대신 predict_proba() 를 써 확률을 직접 받는다. 이 작업은 [그림 1] 에서 검은 세로선(임계값)을 왼쪽으로 옮기는 것과 같다.

```
import numpy as np

# 양성(악성)일 확률
proba = knn.predict_proba(X_test_s)[: , 1]

for thr in [0.3, 0.4, 0.5, 0.6, 0.7]:
    y_thr = (proba >= thr).astype(int)
    p = precision_score(y_test, y_thr, zero_division=0)
    r = recall_score(y_test, y_thr, zero_division=0)
    print(f"thr={thr:.1f} precision={p:.3f} recall={r:.3f}")
```

임계값을 낮출수록 재현율은 올라가고 정밀도는 떨어지는 트레이드오프를 직접 확인할 수 있다.

[7 단계] ROC 곡선과 AUC

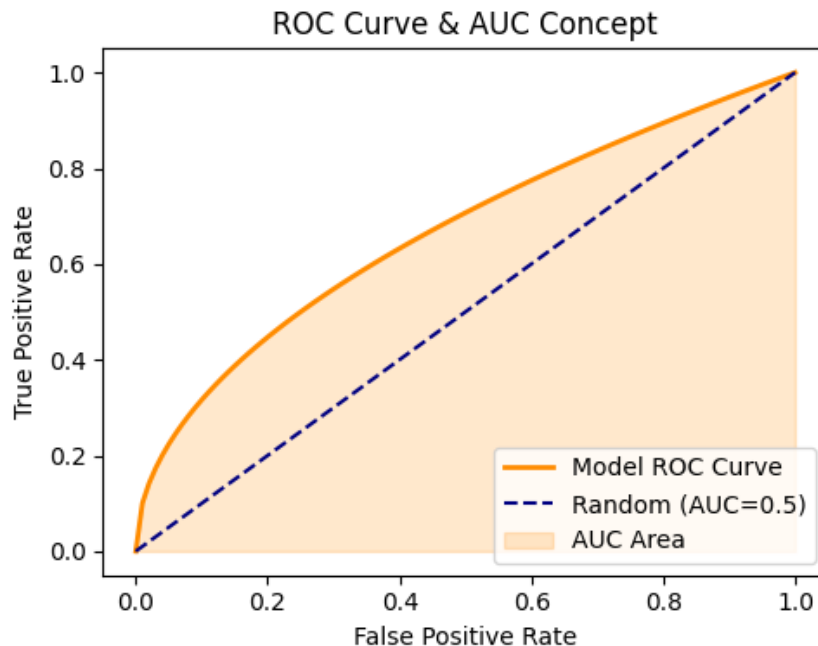
```
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt

fpr, tpr, thr = roc_curve(y_test, proba)
auc = roc_auc_score(y_test, proba)

plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, label=f"KNN (AUC={auc:.3f})")
plt.plot([0, 1], [0, 1], "--", color="gray", label="무작위")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Breast Cancer / KNN")
```

```
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

실행하면 [그림 4] 와 비슷한 ROC 곡선이 나온다. 곡선이 좌상단에 얼마나 가까이 붙는지가 모델의 분별력을 보여준다.



💡 [코멘트] AUC 는 곡선 아래의 면적(Area Under Curve)을 의미한다. AUC 가 1 에 가까울수록 완벽한 모델이고, 0.5 에 가까울수록 동전 던지기처럼 무작위로 찍는 모델을 뜻한다. (보통 0.8 이상이면 준수한 모델로 평가함.)

[8 단계] PR(Precision-Recall) 곡선과 AP

```
from sklearn.metrics import precision_recall_curve, average_precision_score

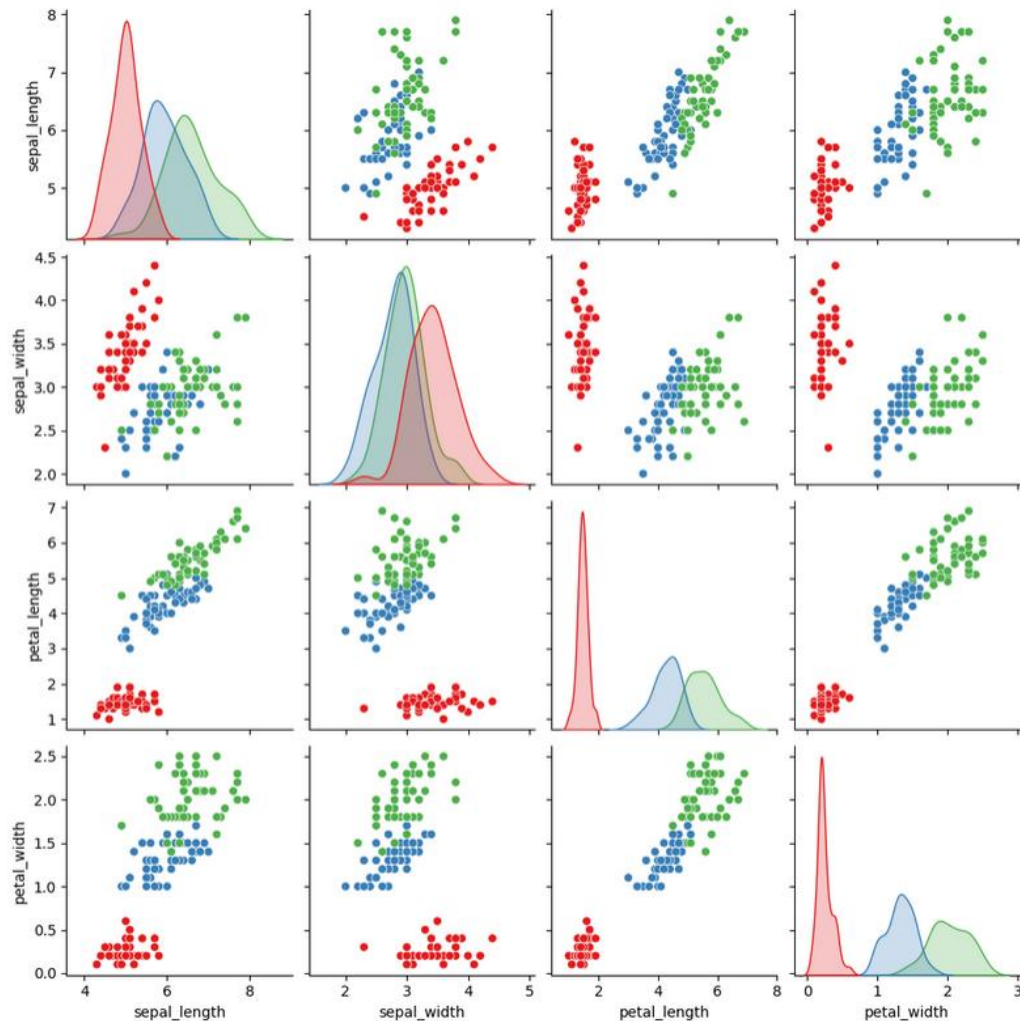
prec_arr, rec_arr, thr_arr = precision_recall_curve(y_test, proba)
ap = average_precision_score(y_test, proba)

plt.figure(figsize=(6, 5))
plt.plot(rec_arr, prec_arr, label=f"KNN (AP={ap:.3f})")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

[9 단계] 다중 분류 평가 – Iris 로 확장

이진 분류에서 익힌 지표들을 다중 분류로 확장해본다. iris(붓꽃) 데이터셋은 세 품종(setosa, versicolor, virginica)을 4 개의 측정값으로 분류하는 대표적인 다중 분류 예제다.





[그림 6] Iris 데이터셋의 산점도 행렬 — 네 가지 측정값(꽃받침 길이/너비, 꽃잎 길이/너비) 사이의 관계를 한눈에 보여준다.

```
iris = sns.load_dataset('iris')
sns.pairplot(iris, hue='species', palette='Set1')
```

색깔로 세 품종이 구분되며, 특히 **꽃잎 길이/너비 쌍**은 품종을 매우 잘 분리한다. 빨간색(Setosa)은 모든 그래프에서 초록색과 파란색으로부터 멀리 떨어져 있다. 즉, **꽃잎(Petal)의 길이나 너비만 봐도 Setosa를 쉽게 분류할 수 있다**는 뜻이다. 반면 Versicolor와 Virginica는 일부 구간에서 데이터가 겹치는 양상을 보인다.

Petal.Length(꽃잎 길이)와 Petal.Width(꽃잎 너비)의 관계를 보면, 점들이 우상향하는 직선 형태로 조밀하게 모여 있다. 이는 꽃잎이 길어질수록 너비도 비례해서 넓어진다는 강력한 **양의 상관관계**를 의미합니다.

Sepal.Length와 Sepal.Width 사이의 그래프를 보면 점들이 넓게 퍼져 있습니다. 이는 꽃받침의 길이와 너비 사이에는 뚜렷한 선형적 관계가 부족함을 시사합니다.

변수 선택: 어떤 특성(Feature)이 품종을 구분하는 데 가장 결정적인 역할을 하는지(여기서는 꽃잎 정보) 알 수 있습니다.

```
from sklearn.datasets import load_iris
from sklearn.metrics import classification_report, f1_score
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris()
X_tr, X_te, y_tr, y_te = train_test_split(
    iris.data, iris.target, test_size=0.2, stratify=iris.target, random_state=42
)

knn3 = KNeighborsClassifier(n_neighbors=5).fit(X_tr, y_tr)
y_hat = knn3.predict(X_te)
```

```
print("macro F1   :", f1_score(y_te, y_hat, average="macro"))
print("micro F1   :", f1_score(y_te, y_hat, average="micro"))
print("weighted F1:", f1_score(y_te, y_hat, average="weighted"))

print(classification_report(y_te, y_hat, target_names=iris.target_names))
```

Iris 는 클래스 분포가 균형이라 macro·micro·weighted 셋이 거의 비슷하게 나온다. 클래스 불균형이 있는 데이터에서는 셋의 차이가 커지므로, 어떤 평균을 쓰는지 명시하는 습관이 필요하다.

7. 오늘 수업 정리

- 정확도는 직관적이지만 클래스 불균형 상황에서는 신뢰할 수 없다. 항상 데이터 분포를 먼저 본다.
- 혼동 행렬의 TP·FP·FN·TN 은 모든 분류 평가 지표의 출발점이다.
- 정밀도는 "양성이라 외친 것의 신뢰도", 재현율은 "진짜 양성을 놓치지 않는 정도"이며, 보통 트레이드오프 관계다.
- F1 은 두 지표의 조화평균으로, 한쪽만 잘하는 모델을 걸러낸다.
- 다중 분류에서는 macro/micro/weighted 중 무엇을 쓰는지 항상 명시한다. 클래스 불균형에선 macro 를 우선 본다.
- predict_proba()로 확률을 받아 임계값을 조정하면 도메인 요구에 맞춰 정밀도-재현율 균형을 직접 설계할 수 있다.
- ROC-AUC 는 임계값과 무관한 모델 자체의 분별력을, PR-AUC 는 불균형 데이터에서 실용적 성능을 보여준다.

과제

- breast_cancer 데이터에서 KNN 의 n_neighbors 를 1, 3, 5, 7, 11 로 바꾸며 정밀도·재현율·F1·AUC 를 표/그래프로 정리한다. 어떤 k 값이 의료 도메인에 가장 적합한지 본인의 의견과 근거를 쓴다.
- predict_proba 의 임계값을 0.1 부터 0.9 까지 0.1 단위로 바꾸며 정밀도-재현율 곡선을 직접 그려보고, "악성 재현율 95% 이상을 보장하면서 정밀도가 가장 높은" 임계값을 찾는다.
- 불균형이 심한 데이터(예: sklearn.datasets.fetch_covtype 의 일부 또는 imblearn 예제 데이터)에서 정확도와 F1·PR-AUC 가 얼마나 차이 나는지 비교 보고서를 짧게 작성한다.